Lecture 1 · Part I · Lecture notes

What machine learning is, and how we will work

Dataset: California housing

How to use these notes

The primary text for this lecture is Géron, Chapters 1–2. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 A day with machine learning **2**

2 The brief **3**

2.1 The first question is never technical . . . 3

2.2 What the current solution is . . . 3

2.3 Framing, in four questions . . . 4

3 Looking at the data, once **4**

3.1 What `info()` reports . . . 4

3.1.1 207 districts are incomplete . . . 4

3.1.2 One attribute is not numerical . . . 5

3.2 Summary statistics . . . 5

3.3 Four things the histograms show . . . 5

3.3.1 Observation 1 — the income is not in dollars . . . 5

3.3.2 Observation 2 — the target is capped . . . 5

3.3.3 Observation 3 — the scales differ wildly 6
3.3.4 Observation 4 — many attributes are right-skewed 6

4 Stop: measuring honestly **6**

5 The notebook **7**

6 Design exercise: a property marketplace's instant home valuation **7**

7 Exercises **9**

Summary **9**

---

1 A day with machine learning

The lecture opens with a day rather than a definition: fifty-one applications of machine learning that a student meets between waking and sleeping, one slide each, from the phone that finished charging just before the 06:30 alarm because it has learned when you get up, to the episode that plays next at 23:30, its cover picture chosen for you. Face unlock, the spam filter, the suggested reply, the feed, the arrival time, the camera at the restricted-traffic gate, camera translation, search, the keyboard's next word, live captions, the robot in the warehouse, the radiologist's second reader, a protein folded on a computer. None of them is explained; the point is how many there are, and that almost none of them announces itself.

A few ideas run through the list, and the rest of the course picks each of them up:

- **Much of it is prediction, not measurement.** An arrival time, tomorrow's demand for yoghurt in one shop, the evening peak on the electricity grid, the next word you will type: each is a guess about something that has not happened yet, and it is judged by how often it is right.
- **Errors come in two kinds, and they do not cost the same.** Emergency braking can brake for nothing or fail to brake; a fraud system can block your card abroad or miss the thief; a smartwatch can raise a false alarm about your heart or stay silent. Which error a system should prefer is a question about costs, not about the model.
- **Right most of the time is not right.** Autocorrect fixes the typo you missed and then, with complete confidence, rewrites your message to your professor; a chat assistant says something false in exactly the voice it uses for something true; the code assistant is usually right, and this course is about the rest of the time. Evaluation is the spine of the course for that reason.
- **Labelled data is the scarce resource, and somebody has to make it.** The puzzles that ask you to select every traffic light have, at times, been labelling pictures for a model.
- **Some uses are a question of law.** Ranking job applications and scoring credit are high-risk

uses under the EU's AI Act. A recruiting model that learned from a history of mostly male hires learned to penalise the word "women's", and was scrapped.

Many of the applications come back with a lecture of their own: image classification in Lecture 12, detection in Lecture 14, forecasting in Lecture 15, chat assistants in Lecture 18, search in Lectures 19 and 20, recommendation in Lectures 21 and 22, text and images together in Lecture 23, and generation in Lecture 24.

The last slide of the day is the bridge into the brief. A property website shows an estimated price for almost any home, including one nobody is selling, and that estimate is the model this lecture and the next start building. It is also a reminder that an error is not just a number: one company bought homes on the strength of its own estimates, overpaid for thousands of them, and shut the business down.

2 The brief

You are a data scientist at a real-estate investment company. The task: use California census data to build a model of housing prices. The data gives population, median income and median housing price for each block group — the smallest geographical unit the US Census publishes, which we will call a **district**. The model predicts a district's median housing price from the other metrics.

2.1 The first question is never technical

What is the business objective? Building a model is not a goal. How the company uses the output determines the framing, the algorithm, the metric, and how much effort is worth spending.

The answer here: the prediction is fed to *another* machine learning system, together with other signals, to decide whether a district is worth investing in. A piece of information fed to a downstream system is called a **signal**, after Shannon.

The consequence of being a component

Nobody downstream ever looks at your prediction. A component that quietly degrades inside a chain of components is not noticed by anyone — which is exactly why the evidence has to come from you, unprompted, rather than from a complaint.

2.2 What the current solution is

District prices are currently estimated manually by experts: a team gathers information and applies complex rules. When they can check their estimates against the truth, they are typically off by more than 30%.

Read that figure carefully, because two things about it matter later. It is a *relative* criterion — 30% of the district's price — and in the next lecture we choose a metric measured in dollars, which is not the same thing. It is also the stakeholder's claim about their own team, not a measurement we made. We will treat it as an assumption and label it as one wherever it appears.

2.3 Framing, in four questions

Before any code: what kind of supervision does this need; is it classification, regression, or something else; batch or online learning; and what is everyone assuming? The first three take a minute.

Question	Answer	Because
Supervision	supervised	every district carries its expected output
Task	multiple, univariate regression	predict one value, from several features
Learning mode	batch, offline	no data stream, and 20,640 rows fit in memory

"Multiple" refers to the several input features; "univariate" to the single output. Predict several values per district and it becomes multivariate regression; add a data stream and it becomes online learning. The other kinds of supervision, for reference: **unsupervised** (no labels, so structure has to come from the inputs alone), **semi-supervised** (a few labelled instances among many unlabelled), **self-supervised** (labels manufactured from the data itself — mask a word, predict it) and **reinforcement** (no labels, a reward signal for a sequence of actions).

The fourth question is the one that can sink the project: *what is everyone assuming?*

Failure condition — the assumption that would have sunk this project

Suppose the downstream system converts our prices into categories — cheap, medium, expensive — and discards the numbers. Then this is a *classification* problem, every regression metric we are about to choose is the wrong one, and we would have built the wrong system competently.
We asked. They need the actual prices. We may proceed — and the value of the question is not that the answer was reassuring, but that it was cheap and the alternative was months.

3 Looking at the data, once

The data is fetched by a function rather than by hand: it downloads the tarball if it is absent and reads the CSV if it is present, because the data will change and you will need it again on another machine. Ten attributes, one row per district. The habit is `head()` then `info()`, always, before anything else.

3.1 What `info()` reports

`RangeIndex: 20640 entries`, ten columns, nine of them `float64` and one `object`. Two facts fall straight out of it.

207 districts are incomplete

`total_bedrooms` has 20,433 non-null values against 20,640 rows. Three options, all legitimate: drop those districts, drop the whole attribute, or fill the gaps with a chosen value. We return to this in the build, and the choice is not obvious — dropping 207 rows is cheap, dropping a column

that correlates with the target is not, and filling requires a decision about *what* to fill with and *on which rows the filling value is computed*. That last clause matters as soon as the data is split: from then on, anything learned from it is learned from the training set alone.

One attribute is not numerical

`ocean_proximity` takes repeated values from a small set: this is a categorical attribute, not free text.

Category	Districts
`<1H OCEAN`	9,136
`INLAND`	6,551
`NEAR OCEAN`	2,658
`NEAR BAY`	2,290
`ISLAND`	5

Note `ISLAND`: five districts in the whole state. Rare categories cause trouble later — a one-hot column that is nonzero five times in 20,640 rows can appear in a training fold and not in a validation fold, and a stratified split cannot protect a stratum that small.

3.2 Summary statistics

`describe()` gives a numerical summary of every numeric column. Three things in it need reading correctly. `count` ignores nulls, which is why `total_bedrooms` reads 20,433. `std` is the standard deviation, how dispersed the values are. And the rows `25%`, `50%` and `75%` are the quartiles, the middle one being the median. For a bell-shaped distribution the 68–95–99.7 rule applies: about 68% of values fall within 1σ of the mean, 95% within 2σ and 99.7% within 3σ. Most of the columns here are not bell-shaped, which is what the histograms show next.

3.3 Four things the histograms show

Plotting all nine numeric attributes at fifty bins takes one line and repays it immediately.

Observation 1 — the income is not in dollars

`median_income` runs from roughly 0.5 to 15. That is not a salary. The data was scaled and capped: at 15 (in fact 15.0001) at the top and 0.5 (0.4999) at the bottom. The units are roughly tens of thousands of dollars, so 3 means about $30,000. Preprocessed attributes are normal and not a defect; understanding how they were computed is not optional.

Observation 2 — the target is capped

`median_house_value` is capped, and it is our *label*.

Failure condition — why a capped label is more serious than a capped feature

The model will learn that prices never exceed the cap, because in the training data they never do. If the client needs accurate predictions above $500,000, the model is not merely inaccurate there — it is structurally incapable of producing such a number.
Two honest fixes: collect proper labels for the capped districts, or remove them from *both* the training and the test set. Removing them from the training set alone would leave you scoring a model on districts it was forbidden to learn about.

The spike is not small: 965 districts, 4.7% of all 20,640, sit at the cap, $500,001, and carry a label that is a ceiling rather than a price.

Observation 3 — the scales differ wildly

`total_rooms` runs from 2 to 39,320; `median_income` from 0.4999 to 15.0001. Whether that matters depends entirely on the method.

Scaling is required for	Scaling does nothing for
gradient descent, logistic regression, SVMs, k-nearest neighbours, k-means, PCA, neural networks	decision trees, random forests, ordinary least squares

The left column is anything that measures a distance, penalises a coefficient, or takes a step. The right column has reasons: a tree splits on one feature at a time, so any monotone rescaling produces the identical tree; and least squares is equivariant under an invertible affine map of the features — the fit is identical and the coefficients simply absorb the change.

Linear regression, trees and forests — the models this dataset will be fitted with — are all in the right-hand column, and that is not an incidental remark. It is why the scale-before-split leak, which is the canonical beginner's error, will cost *nothing* measurable here: an invertible affine map, applied to estimators that do not care. That is a claim to be measured, not asserted.

Observation 4 — many attributes are right-skewed

They extend much further to the right of the median than to the left, which makes patterns harder for some algorithms to detect. The usual repair is a transform toward symmetry: a square root or a logarithm. A feature following a power law is a good candidate for log — districts of 10,000 people are about ten times rarer than districts of 1,000, not exponentially rarer, and the log of such a variable is roughly linear in its rank.

4 Stop: measuring honestly

Four observations from four histograms, and we have not yet plotted one variable against another. That is exactly the moment to stop.

The only way to know how a model generalises is to try it on cases it has never seen. So the data is split into a **training set** and a **test set**, and the error rate on new cases is the **generalisation**

error. A low training error with a high generalisation error is **overfitting**: the model has learned the training set, noise included. The split is typically 80/20, though the proportion is a function of size — with ten million instances, holding out 1% is plenty.

The lecture ends here, deliberately before the split. Lecture 2 starts with it: why the test set has to be set aside *before* any further exploration, and how to make sure it represents the data.

5 The notebook

The Lecture 1 notebook runs on Colab's free CPU tier and contains everything above that touches the data, in order: the fetch, `info()`, the missing values and the categories, the histograms, the range of every column, and the cap — counted on all 20,640 districts. Every code cell carries the specification that would produce it.

What to change

Run it top to bottom once. Then change one thing — `bins=50` to the default `bins=10` in the histogram cell — and look for the two spikes. One argument decides whether you ever see the cap: at ten bins it shares a bar with everything just below it.

6 Design exercise: a property marketplace's instant home valuation

The lecture closes with a design exercise in the manner of an ML system-design interview, on the question the day's last slide raised.

The task. A property marketplace — the app where homes are listed and searched — wants an instant valuation on every listing and on the page an owner sees for their own home. Buyers read it before offering, owners before selling, and the agents who pay the marketplace for leads get it with each lead. The business measures leads sold and owners who go on to list; a wrong number costs a listing, a buyer, or a visit from the advertising regulator. The question put to the data team is not which model but whether a model is the right answer at all. Given: the public record of past sales — address, date, price — joined to what is known of each home, and the asking prices of current listings. Tens of millions of homes, tens of thousands of sales a month, a few a year in a small town. The output feeds the visitor who reads it, the search ranking, and the agents who buy the leads. No model is at the centre yet: a framing, a baseline, a label, one look at the data and a held-out set are everything that comes before a model.

1. **Frame it.** Building a model is not the goal: the marketplace wants owners to list and buyers to enquire, and how the valuation is used decides the framing, the metric and the effort worth spending. Every sale carries its price, so supervised; one number out, so regression; sales arrive monthly and fit in memory, so batch; and the fourth question is what everyone is assuming. *Pitfall:* the search team uses only whether the valuation is above or below the asking price, to badge a listing "good value", so for that consumer the system is a classifier and every regression metric measures the wrong thing. Ask each consumer's team: the visitor needs the price, the badge the band, and a price can be turned into a band but not the reverse — so the output is the price, the band is derived downstream, and the answer is

written down before any code.

2. **The baseline.** The page today shows the last recorded sale price moved by the region's index, or an agent's valuation: a lookup, no model, computed in a line, and the baseline any model must beat on sales neither has seen. The product manager's "usually within a tenth of the sale price" is a relative claim, not a measurement. *Pitfall:* without the lookup's error on the held-out sales, any error the model reports looks like progress, the release is approved on it, and "within a tenth" becomes marketing copy. Measure the lookup first, on exactly the sales the model will be measured on, and label the figure an assumption wherever it appears.

3. **The label is a sale, not a wish.** One row per sale: the price paid, the date, and what was known about the home on that date. Asking prices are plentiful and are not the label; homes that never sold have no label at all. *Pitfall:* training on asking prices because there are ten listings for every sale. The model learns what sellers hope for, which runs above what buyers pay by a margin that varies by town and season; owners list too high on its advice, and the error against real sales is never seen because the test set is wishes too. Label with recorded sales only, and keep the asking price as a feature at most.

4. **Look at the whole table once.** `head()`, `info()`, `describe()` and a histogram of every column: which columns are incomplete, which are categorical and carry a rare level, how far apart the scales are, how skewed the prices, and where values pile up at one point. *Pitfall:* the public record truncates prices above a reporting threshold, so the most expensive sales all carry the same value; a model learns that prices never exceed it, and the error on exactly the homes with the richest leads is invisible because the held-out sales are capped too. Count the spike, then obtain the true prices or remove the capped sales from *both* sets, and say so.

5. **Hold out before looking closer.** The only way to know how a valuation generalises is to try it on sales it has never seen: split into a training set and a test set before one variable is plotted against another, with a share that follows the size of the data. Low training error with high held-out error is overfitting, and the training error alone can never show it. *Pitfall:* a home sold twice is in the record twice, and a random split puts one sale on each side; the second is not a case the model has never seen, and the error the release rests on reads better than it is. Hold out homes, not sales, and check that every address is on one side only.

6. **The metric.** The error is one number computed on the held-out sales. Measured in money, a mistake of twenty thousand on a flat and on a mansion count the same; the product team's criterion is a share of the price. *Pitfall:* the model is tuned on an error in money, dominated by the expensive homes, and reported to a team whose criterion is relative; the two can move in opposite directions, better on the mansions and worse on the flats that are most of the traffic. Write the metric beside the criterion before anything is fitted, and report the error in both forms and by price band, so that nobody approves a release on the average.

7. **When a rule or a lookup table is the better answer.** A home that sold recently or carries a fresh valuation gets the record moved by the index: a rule. A town with a handful of sales a year gets the area's median price per square metre, by type and size band, refreshed monthly: a table. A city with thousands of sales and prices that change street by street gets a model — if it beats the table on held-out sales by a margin the visitor can feel. *Pitfall:* a model is built because the roadmap said so; in the small towns it is fitted on a dozen rows, overfits them and loses to the table, and nobody sees it because the national average is dominated by the

cities, while the owners there notice. Decide per area, and ship the model only where it is measured to win.

8. **After launch.** The held-out error was measured on last year's sales, and prices drift; batch learning on a schedule refits on the new sales each month, re-measures on the sales closed since, and only then goes live, with the data team on call for the error by region. The number also changes the market it predicts: owners set asking prices from it. *Pitfall:* an error measured once is quoted for years while the site's own number steers the asking prices that become the next sales — one marketplace bought homes on the strength of its own valuations, overpaid for thousands of them, and closed the business. Keep measuring on the sales that close after each release, record what the site showed on the day of each sale so that its pull can be seen, and report the error month by month.

9. **Ship it.** Objective agreed with each consumer → the lookup measured as the baseline → recorded sales as the label → one look at the whole table → homes held out before exploring → the metric written beside the criterion → rule, table or model, decided per area → measured on the sales that close after launch → refit monthly.

7 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 2's deck and in the exercise book.

1. The brief says the output feeds a downstream system that expects a price. Name the one thing this fact settles about the problem framing, and one thing it leaves open. [4]

2. `info()` reports that 207 of the 20,640 districts have no value for `total_bedrooms`. Give the three legitimate ways of handling the incomplete districts, and say which of them still leaves a decision to make. [4]

3. `total_rooms` runs from single digits to tens of thousands, `median_income` from about 0.5 to 15. Name two methods for which this difference in scale matters and two for which it does not, and give the reason for one of the latter. [5]

4. The median income column is capped at 15. Give one consequence for the *model* and one for the *evaluation*, and say which of the two you would raise with the client first. [5]

5. A model's error is low on its training set and high on new districts. Name the condition, and explain why the training error alone could never have revealed it. [4]

Summary

The lecture began with a day of machine learning: fifty-one applications already in daily use, from the phone that unlocks to the episode that plays next, most of them predictions, many with two kinds of error that cost different amounts, and none of them right every time — which is why evaluation runs through the whole course.

The brief was framed before any code: supervised, multiple univariate regression, batch — and the assumption that would have made it a classification problem was checked with the downstream team rather than guessed. The data was looked at once, in full, which surfaced 207 incomplete districts, a categorical attribute with a five-district level, an income that is scaled and capped, a target at the cap, $500,001, for 965 districts, scales spanning four orders of magnitude, and pervasive right skew.

Then the lecture stopped, before plotting one variable against another: the generalisation error is measured on cases the model has never seen, so the test set has to exist before the exploration does.

The closing design exercise asked whether a property marketplace's instant home valuation should be machine learning at all, in nine steps — the objective agreed with each consumer, a lookup measured as the baseline, recorded sales as the label, one look at the table, homes held out before exploring, the metric written beside the criterion, a rule or a table wherever they win, and the error re-measured after launch — each step named for the pitfall it avoids.